



Carrera:
Licenciatura en
Ing. En Programación y Web master

Docente:
L.I.S.C. Mario Alonso Segovia Gutiérrez

Materia:
Programación Avanzada

Alumno(s):
Pedro José Marín Acosta

Fecha:
19 de julio del 2026

Cuadro Comparativo

Diferencias entre sesiones, cookies y JSON Web Tokens

Aspectos	Sesiones	Cookies	JSON Web Tokens
Definición	Es el proceso por el cual un servidor mantiene el estado de una entidad (usuario) que interactúa con él, permitiendo que el backend recuerde cómo reaccionar a solicitudes subsiguientes a lo largo de una transacción.	Mecanismo de almacenamiento o del lado del cliente más utilizado para la gestión de sesiones en navegadores	Es un método estándar de la industria (abierto y definido en el RFC 7519) para representar de forma segura declaraciones o reclamos (<i>claims</i>) entre dos partes. Permite el intercambio seguro de credenciales de autenticación que el cliente adjunta en cada petición protegida
¿Dónde se almacena la información?	En el Servidor: La información de negocio. En el Cliente (Navegador): Solo se almacena un identificador único y aleatorio conocido como Session ID (ID de sesión).	En el Servidor: No se almacena nada. En el Cliente (Navegador): Toda la información de la cookie se guarda en el almacenamiento local del propio navegador web.	En el Servidor: Por definición estricta, no se almacena nada. El servidor solo posee la <i>clave secreta</i> para verificar que el token que reciba sea auténtico. En el Cliente (Navegador): Toda la información del usuario (<i>claims</i>) viaja dentro del string del token y se guarda en el cliente. OWASP y MDN señalan que el desarrollador debe elegir entre dos lugares para guardarlo en el navegador: Web Storage (localStorage / sessionStorage). Una Cookie HTTP con atributo HttpOnly.
¿Quién administra la información	El cliente solo funciona como un portador pasivo del identificador. El servidor mantiene el	En las cookies, la administración y el	En los JSON Web Tokens (JWT), la información es administrada y custodiada físicamente

(cliente o servidor)?	estado y asocia de forma activa ese identificador con los datos del usuario alojados en su infraestructura	almacenamiento o físico de la información ocurren en el cliente pero el control, la lógica de negocio y la creación son dirigidos prioritariamente por el servidor. Es un modelo híbrido basado en el ciclo de vida de la petición HTTP.	por el cliente en forma de un token sin estado (stateless), pero su validez, integridad y generación son controladas estrictamente por el servidor mediante criptografía.
Nivel de seguridad	PHP cuenta con un mecanismo nativo de gestión de estado, pero advierte que su seguridad inicial "deja mucho que desear" si se mantiene por defecto. Nivel de Aceptación: Posee un modo permisivo por defecto. Requisitos Obligatorios: Exige activar imperativamente el modo estricto mediante <code>session.use_strict_mode = On</code> para rechazar IDs de sesión no inicializados por el servidor (prevención de <i>Session Fixation</i>). Configuración del Entorno: Requiere obligatoriamente restringir el transporte mediante <code>session.use_only_cookies = On</code>,	Las cookies son <u>inherentemente inseguras</u> si se usan de forma nativa. Su seguridad depende de mitigar ataques mediante la restricción de su alcance con atributos específicos. <u>Mitigación de XSS (Robo de Identidad).</u> <u>Mitigación de MITM (Intercepción de Red).</u> <u>Mitigación de CSRF (Falsificación de Peticiones).</u>	El nivel de seguridad de JSON Web Tokens (JWT) <u>no está determinado por el formato en sí, sino por la rigurosidad de su implementación y manejo.</u> Garantía de integridad, no de privacidad: La especificación oficial aclara que los <u>JWT estándar están firmados, pero no cifrados.</u>

	session.cookie_secure = On (solo HTTPS) y session.cookie_httponly = On (bloqueo de acceso vía JavaScript/XSS).		
Perseverancia de la información	PHP gestiona la persistencia de datos mediante un ciclo de vida automatizado en el servidor: <u>Identificador Único.</u> <u>Almacenamiento Local.</u> <u>Persistencia entre.</u> <u>Peticiones.</u> <u>Serialización.</u>	En el ecosistema web, las cookies perseveran la información basándose en el <u>intercambio continuo de cabeceras HTTP entre el cliente y el servidor.</u> Dado que HTTP es un protocolo sin estado, las cookies actúan como el mecanismo principal para mantener la continuidad de los datos <u>Cookies de sesión:</u> No especifican directivas de tiempo. <u>Cookies persistentes:</u> Utilizan los atributos Expires (fecha concreta UTC) o Max-Age (segundos de vida restantes, el cual tiene	En un JSON Web Token (JWT), la información no se "persevera" en el sentido tradicional de persistencia en una base de datos del servidor, sino que se mantiene de forma autocontenida y apátrida (stateless) en el propio token mediante codificación y se protege criptográficamente contra alteraciones. Estructura de 3 partes (header.payload.Signature). El payload contiene las afirmaciones o claims. Codificación Base64URL. Es auto contenido toda la info esta dentro del propio token.

		prioridad). El navegador las escribe directamente en el almacenamiento físico, transmitiéndolas en cada petición que coincida con el Domain y Path configurados.	
Ventajas	Las ventajas de utilizar <u>sesiones</u> del lado del servidor (stateful), se centran principalmente en la seguridad robusta, el control absoluto del estado y la simplicidad en la persistencia de datos.	Las <u>cookies</u> permiten recordar información de estado entre peticiones HTTP independientes. Gestión de sesiones. Personalización de usuario. Rastreo y Análisis. Persistencia del lado del cliente.	<u>Tamaño compacto y portabilidad.</u> <u>Autocontenido.</u> <u>Firmado criptográfico robusto.</u>
Desventajas	Las principales desventajas del uso de sesiones tradicionales (basadas en servidor y cookies) se centran en la falta de escalabilidad horizontal, la vulnerabilidad a ataques de seguridad específicos y el consumo constante de recursos en el servidor.	Las cookies presentan diversas desventajas de seguridad y técnicas, especialmente al almacenar información confidencial o al integrarse con arquitecturas	Dificultad para la Revocación Inmediata. Falta de invalidación nativa. Problemas en flujos comunes: resalta que un JWT no se puede invalidar de manera inmediata por sí mismo durante un cierre de sesión, cambios de contraseña, revocación

	Acoplamiento al servidor. Complejidad en sistemas distribuidos. Contraste con JWT. Almacenamiento persistente en el servidor. Gasto en recolección de basura en el servidor. Vulnerable a ataques CSRF (Cross-Site Request Forgery). Secuestro de Sesión (Session Hijacking). Fijación de Sesión. Bloqueo de archivos (Concurrencia).	de tokens como JWT. Secuestro de Sesión (Session Hijacking). Exposición en tránsito. Manipulación del lado del cliente. Vulnerable a ataques CSRF (Cross-Site Request Forgery). Impacto en el rendimiento (Ancho de banda). Capacidad de almacenamiento o muy limitada. Rastreo invasivo.	de permisos o desactivación de cuentas. Complejidad añadida: Para mitigar esto, se deben implementar listas de bloqueo o bases de datos de tokens revocados. No son cifrados por defecto. Visibilidad pública. Riesgo de fugas. Ataques al algoritmo "none". Confusión de algoritmos. Uso incorrecto de funciones de código. Incremento del ancho de banda. Impacto en el rendimiento. Vulnerabilidad a XSS. Vulnerabilidad a CSRF.
Casos de uso	Aplicaciones web tradicionales. Sistemas con revocación inmediata. Aplicaciones de alta sensibilidad.	Persistencia de preferencias. Almacenamiento seguro de credenciales. Seguimiento y analítica.	Autenticación en APIs y Microservicios. Sistemas de Single Sign-On (SSO). Aplicaciones móviles

Ejemplo de aplicaciones donde se utilizan	Banca en línea. Paneles de administración empresariales. Foros y comunidades tradicionales.	Carritos de compras en e-commerce. Preferencias de interfaz. Rastreo y analítica (Marketing).	Aplicaciones Móviles (Spotify, Instagram). Arquitectura de Microservicios. APIs Públicas y SaaS.
Riesgos de seguridad más comunes	El manejo de sesiones depende de un identificador único que vincula al navegador con el estado guardado en el servidor. Sus riesgos implican la gestión y el ciclo de vida de este identificador. <u>Secuestro de Sesión (Session Hijacking).</u> <u>Fijación de Sesión (Session Fixation).</u> <u>Exposición del ID de Sesión en URLs.</u> <u>Persistencia de Sesiones Obsoletas.</u>	Las cookies son el mecanismo de almacenamiento en el navegador. Sus riesgos están ligados a la falta de restricciones en el acceso a los datos desde el cliente o en el envío automático de peticiones. Robo de Cookies mediante XSS. Fuga de datos por canal inseguro. Falsificación de Peticiones en Sitios Cruzados (CSRF). Falta de aislamiento de contexto (SameSite ausente).	Al ser un mecanismo de autenticación sin estado donde el token viaja con los datos del usuario, los riesgos se trasladan a la validación criptográfica y a la lógica de su almacenamiento. Aceptación del Algoritmo None. Ataque de Confusión de Algoritmos (HMAC vs RSA). Exposición en Web Storage. Imposibilidad de Revocación Inmediata. Uso de Claves Secretas Débiles.

medidas recomendadas as para proteger la información	La seguridad en sesiones se gestiona en el servidor para evitar que un atacante suplante a un usuario legítimo. Regenerar el ID. Destrucción total de la sesión al cerrar esta. Tiempo de expiración para sesiones abandonadas. Validación de IP/User-Agent. Almacenamiento seguro, rutas privadas.	Las cookies son el medio de transporte. Deben blindarse para que nadie pueda interceptarlas ni leerlas desde el navegador. Flag HttpOnly. Flag Secure. Atributo SameSite. Prefijos de seguridad. Expiración estricta.	Los JWT son tokens auto contenidos. Su seguridad radica en la validez de su firma y en dónde se guardan. Prohibir algoritmo "none". Algoritmos robustos: Utiliza firmas asimétricas como RS256 o firmas simétricas fuertes con claves de alta entropía (HS256). Cero datos sensibles. Expiración corta. Uso de Refresh Tokens. Validación de Claims: iss & aud.
---	---	---	--

Lista de referencias electrónicas.

JWT en Cookies: seguridad de sesiones en aplicaciones escalables - Meta empresa argentina de desarrollo de software. (2026, marzo 9). *MetaOK empresa argentina de desarrollo de software a medida*. <https://www.metaok.com/blog/jwt-en-cookies-seguridad-de-sesiones-en-aplicaciones-escalables/>

Alam, A. (2025, mayo 27). *What are sessions in PHP?* Intellipaat. <https://intellipaat.com/blog/php-sessions/>

de la Rosa, J. M. (2025, enero 29). Implementa sesiones seguras en PHP fácilmente. *SidmacTI*. <https://sidmac.com/blog/implementa-sesiones-seguras-en-php-facilmente>

HTTP cookies. (s/f). MDN Web Docs. Recuperado el 20 de julio de 2026, de <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Cookies>

Rustom, J.-P. (2024, febrero 24). *JWT explained in 4 minutes (With Visuals)*. DEV Community. <https://dev.to/jaypmedia/jwt-explained-in-4-minutes-with-visuals-g3n>

Secure cookie configuration. (s/f). MDN Web Docs. Recuperado el 20 de julio de 2026, de https://developer.mozilla.org/en-US/docs/Web/Security/Practical_implementation_guides/Cookies

Sessions and security. (s/f). Php.net. Recuperado el 20 de julio de 2026, de <https://www.php.net/manual/en/session.security.php>

Babu, S. (2023, septiembre 21). *JWT vs. OAuth:Cuál es bueno para la seguridad web definitiva*. Geekflare Spain. <https://geekflare.com/es/jwt-vs-oauth/>

WSTG - v4.1. (s/f). Owasp.org. Recuperado el 20 de julio de 2026, de <https://owasp.org/www-project-web-security-testing-guide/v41/4->

Web Application Security Testing/06-Session Management Testing/01-
Testing for Session Management Schema

(S/f). Tutsplus.com. Recuperado el 20 de julio de 2026, de
<https://code.tutsplus.com/es/how-to-use-sessions-and-session-variables-in-php--cms-31839t>

3.2 sesiones. (s/f). DWES DOCS. Recuperado el 20 de julio de 2026, de
<https://iescelia.org/docs/dwes/site/cookies-sesiones-seguridad/sesiones.html>

Sesiones (PHP). (s/f). HackMD. Recuperado el 20 de julio de 2026, de
<https://hackmd.io/@bd1-NGPOSje0xW9Pil2fSQ/ryS2i22vi>

(S/f). Programacionymas.com. Recuperado el 20 de julio de 2026, de
<https://programacionymas.com/blog/cookies-y-sesiones>

Bibliografía

Auth. (2024, noviembre 30). *JWT.IO - JSON Web Tokens introduction*. JSON Web Tokens - Jwt.io. <https://www.jwt.io/introduction>

Authentication - OWASP cheat sheet series. (s/f). Owasp.org. Recuperado el 20 de julio de 2026, de
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

WSTG - Latest. (s/f). Owasp.org. Recuperado el 20 de julio de 2026, de
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/02-Testing_for_Cookies_Attributes